

OPERATING SYSTEMS (UE25CS242A)

Unit 1 – Computer-System Architecture, OS Structure & Operations

Computer-System Architecture

Most computer systems use a single general-purpose processor to execute user programs. Along with this main processor, systems also include special-purpose processors.

These special-purpose processors are:

- Device-specific (disk controller, keyboard controller, graphics controller)
- Designed to run a limited and fixed set of instructions
- Low-level components built directly into hardware
- Managed and monitored by the operating system

Example: Disk Controller Microprocessor

- Receives a sequence of disk requests from the CPU
- Maintains its own disk queue
- Implements its own disk scheduling algorithm
- Reduces the overhead on the main CPU by handling disk operations independently

Multiprocessor Systems

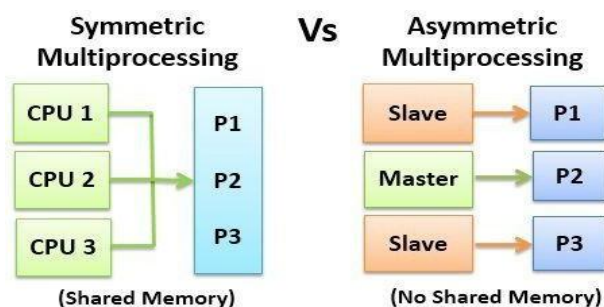
Multiprocessor systems are increasingly common and are also referred to as:

- Parallel systems
- Tightly-coupled systems

Advantages of Multiprocessor Systems

1. Increased Throughput - More processors can complete more tasks in less time.
2. Economy of Scale - Sharing peripherals, memory, and power supplies reduces overall system cost.
3. Increased Reliability - Failure of one processor does not halt the system completely. This is known as graceful degradation or fault tolerance.

Types of Multiprocessor Systems



There are two main types:

1. Asymmetric Multiprocessing (AMP)

- Each processor is assigned a specific task
- One processor may act as the master, others as workers
- Less flexible

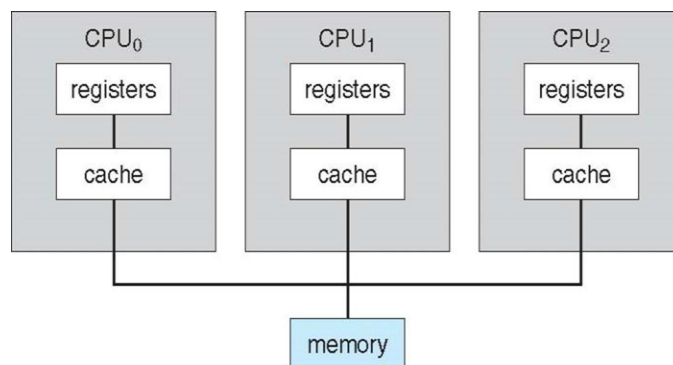
2. Symmetric Multiprocessing (SMP)

- All processors are equal
- Each processor performs all tasks
- Most modern operating systems use SMP

Symmetric Multiprocessing (SMP) Architecture

In an SMP system:

- All processors are peers
- No master–slave or boss–worker relationship
- Each processor has:
 - Its own registers
 - Its own local or private cache
- All processors share the same physical memory



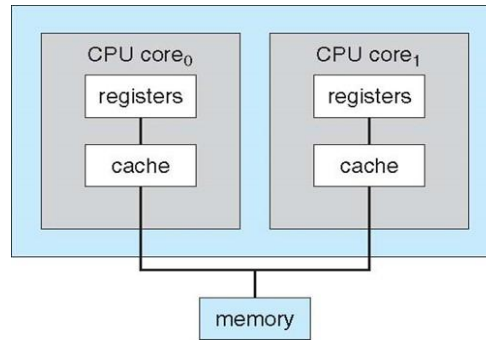
Multi-Core Design

A recent trend in CPU design is **multi-core architecture**, where:

- Multiple processing cores are placed on a **single chip**
- Each core can execute its own instructions independently

Advantages of Multi-Core Systems

- Faster on-chip communication compared to inter-chip communication
- Lower power consumption
- Better performance per watt
- More efficient than multiple single-core processors



Linux Command to Check Core and Cache Details

cat /proc/cpuinfo | more

```

rtn mpx rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaes dtherm ida arat pin pts hwp
0 hwp_notify hwp_act_window hwp_epp nd_clear flush_l1d arch_capabilities
bugs          : spectre_v1 spectre_v2 spec_store_bypass ads
bogomips      : 7200.00
clflush_size  : 64
cache_alignm  : 64
address sizes : 39 bits physical, 48 bits virtual
power manage  :

#lsnang UNIX /dev/pts/5
#-
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: little endian
CPU(s): 16
On-line CPU(s) list: 0-15
Thread(s) per core: 2
Core(s) per socket: 8
Socket(s): 1
NUMA node(s): 1
Vendor ID: GenuineIntel
CPU family: 6
Model: 150
Stepping: 12
CPU MHz: 4704.000
CPU max MHz: 5000.0000
CPU min MHz: 800.0000
bogomips: 7200.00
Virtualization: VT-x
L1d cache: 32K
L1i cache: 32K
L2 cache: 256K
L3 cache: 16384K
NUMA node0 CPU(s): 0-15
Flags: fpu vme de pse tsc mtr pae mce cx8 apic sep mtr pge mca cmov pat pse36 clflush d
ts acpi mpx fixt sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant tsc art arch_perfmon pebs
bts rep_good nopl stopleg nonstop tsc cpuid aperfmperf tsc_known_freq pni pclmulqdq dtherm dtherm
avx_cpl vms vms est tsc2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadli
no_timer sse xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault opb invpcid_single ssbd ibrs
nopl stibp tux_ehadow vmmi f16cpriority ept vpid fsgsbase tsc_adjust bmi1 hle avx2 smep bmi2 erms invp
cid tma mpx rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaes dtherm ida arat pin pts
hwp hwp_notify hwp_act_window hwp_epp nd_clear flush_l1d arch_capabilities
#lsnang UNIX /dev/pts/5
#-

```

Blade Servers

Blade servers are a modern server architecture where:

- Multiple processor boards, I/O boards, and network boards are housed in a single chassis
- Each blade:
 - Boots independently
 - Runs its own operating system

Some blade servers are also multiprocessor systems, making them a combination of:

- Standalone servers
- Multiprocessor machines

In essence, blade servers consist of **multiple independent multiprocessor systems** in one enclosure.

Clustered Systems

Clustered systems consist of multiple independent systems working together.

Key Characteristics

- Usually share storage using a Storage-Area Network (SAN)
- Provide high availability

- Can continue operation even if one system fails

Types of Clustering

1. Asymmetric Clustering

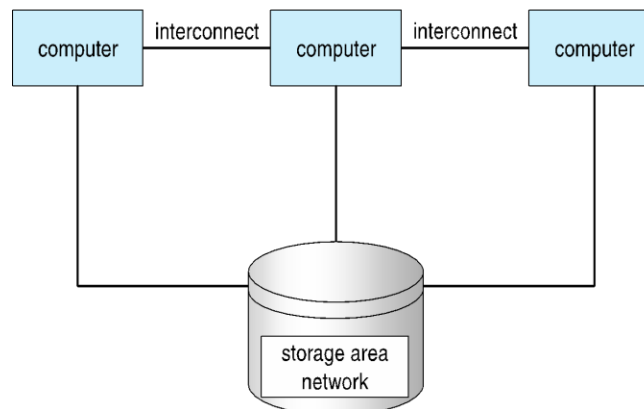
- One machine is in hot-standby mode
- Takes over if the primary machine fails

2. Symmetric Clustering

- Multiple nodes run applications simultaneously
- Nodes monitor each other

High-Performance Computing (HPC) Clusters

- Used for scientific and compute-intensive tasks
- Applications must support parallel execution
- Often use a Distributed Lock Manager (DLM) to prevent data conflicts



Operating-System Structure – Multiprogramming

Multiprogramming is essential for system efficiency.

Why Multiprogramming?

- A single user cannot keep CPU and I/O devices busy at all times
- CPU would be idle during I/O operations

How Multiprogramming Works

- OS organizes multiple jobs (code + data)
- A subset of jobs is kept in main memory
- One job is selected using job scheduling
- When a job waits (e.g., for I/O), OS switches to another job

This ensures the CPU is always utilized.

Operating-System Structure – Multitasking (Time Sharing)

Multitasking is a logical extension of multiprogramming.

Key Characteristics

- CPU switches between jobs very frequently
- Users can interact with programs while they run
- Enables interactive computing
- Response time should typically be less than 1 second

Important Concepts

- Each executing program is called a process
- If multiple processes are ready → CPU scheduling
- If memory is insufficient → Swapping moves processes in and out
- Virtual memory allows execution even if a process is not fully in memory

Operating-System Operations - Operating systems are interrupt-driven.

Types of Interrupts

1. Hardware Interrupt

- Generated by devices (keyboard, disk, network)

2. Software Interrupt

- Also called exception or trap
- Caused by:
 - Errors (division by zero)
 - Requests for OS services (system calls)

OS also handles problems such as:

- Infinite loops
- Unauthorized memory access
- Processes interfering with each other

Dual-Mode and Multimode Operation

Dual-mode operation protects the operating system.

Two Modes

1. User Mode

- User programs execute
- Limited privileges

2. Kernel Mode

- OS executes
- Full access to hardware

A **mode bit** in hardware indicates the current mode:

- 1 → User mode
- 0 → Kernel mode

Privileged Instructions

- Can only be executed in kernel mode
- Prevent misuse of system resources

A system call switches the mode from user to kernel.

Modern CPUs support multi-mode operation, such as:

- Virtual Machine Manager (VMM) mode for virtual machines

Transition from User Mode to Kernel Mode

When:

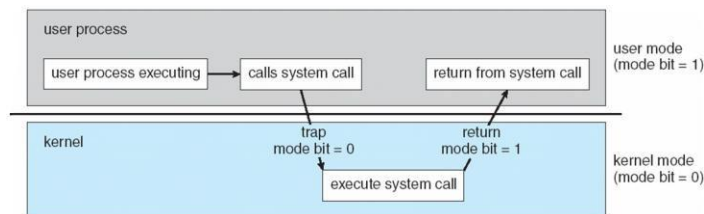
- An interrupt occurs, or
- A trap/system call is issued

The hardware:

- Automatically switches from user mode to kernel mode

After the request is handled:

- Control is returned to user mode
- User program resumes execution safely



Timer

The timer prevents a process from:

- Running indefinitely
- Hogging CPU resources

How Timer Works

- Timer interrupts the system after a fixed or variable interval
- Typical ranges:
 - Fixed: 1/60 second
 - Variable: 1 ms to 1 second

Implementation

- Fixed-rate clock + counter
- OS sets the counter (privileged instruction)
- Counter decrements on each clock tick
- When counter reaches zero → interrupt occurs

Uses of Timer

- Enforces CPU time limits
- Enables preemptive scheduling
- Prevents infinite loops